Calidad de datos — Fuente de verdad

LegalDataPlatform

Humberto Henriquez — Senior Data Engineer / Solutions Architect

Generado 2026-04-24

Data Quality — "Fuente de verdad"

Filosofía

Una plataforma de datos cuyos resultados el negocio no puede confiar es peor que no tener plataforma. Tres principios:

1. **Calidad es un requisito funcional**, no un "nice-to-have" post-hoc.
2. **Reglas como código**, versionadas en git.
3. **Fallar ruidoso y temprano** — mejor un pipeline detenido que datos corruptos en producción.

Las tres líneas de defensa

```
┌──────────────────────────────────────────────────────────┐
│ LINE 1: Structural validation (Pydantic)                 │
│ At ingestion: types, required fields, regex, ranges.     │
│ Rejections → Quarantine. Fast. In-memory.                │
└──────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────┐
│ LINE 2: Business rules (YAML rule engine)                │
│ Post-normalization: cross-field, set membership,         │
│ row count bounds. Fast. Polars-backed.                   │
└──────────────────────────────────────────────────────────┘
                              ↓
┌──────────────────────────────────────────────────────────┐
│ LINE 3: Statistical properties (Great Expectations)      │
│ Post-load or scheduled: drift detection, distributions,  │
│ uniqueness at scale, Data Docs HTML artifact.            │
└──────────────────────────────────────────────────────────┘
```

Line 1 — Pydantic schemas

Qué capturan

- **Tipos** — `amount: Decimal`, `transaction_date: date`.
- **Presencia** — `Field(min_length=1)`.
- **Rangos** — `Field(ge=0)`, validators custom.
- **Regex** — `StringConstraints(pattern= … )`.
- **Normalización** — `to_upper=True`, `strip_whitespace=True`.
- **Coherencia** — `@field_validator` custom (ej. `end_date ≥ start_date`).

Ubicación

src/schemas/legal.py, src/schemas/commercial.py.

Ejemplo

```python
class TransactionSchema(BaseModel):
    amount: Decimal
    currency: Annotated[str, StringConstraints(min_length=3, max_length=3, to_upper=True)]

    @field_validator("amount")
    @classmethod
    def _nonzero(cls, v):
        if v == 0: raise ValueError("amount cannot be zero")
        return v
```

Performance

Pydantic v2 valida ~1M simple models/sec. No es el cuello de botella.

Line 2 — Rule engine declarativo

Motivación

Pydantic vive en los schemas; las reglas de negocio viven en **YAML versionado**. Esto permite que:

- Business analysts propongan reglas en un PR sin saber Python.
- Las reglas se compartan entre pipelines (mismo check en legal y commercial).
- Cambios de calidad tengan historial git (audit trail).

Tipos de reglas soportadas

Tipo	Parámetros	Uso típico
`not_null`	`column`	Columnas obligatorias
`unique`	`column`	IDs, references
`regex`	`column`, `pattern`	Formato de email, hash SHA-256, códigos
`in_set`	`column`, `values`	Taxonomías (statuses, types)
`range`	`column`, `min`, `max`	Bounds numéricos
`row_count_range`	`min`, `max`	Detectar batches vacíos o anómalos

Severidad

```yaml
severity: error     # Bloquea el pipeline (assert_no_errors raises)
severity: warning   # Solo emite métrica y log
```

Ejemplo completo

```yaml
# src/data_quality/rules/transactions.yaml
suite: transactions_silver
rules:
  - name: amount_not_null
    type: not_null
    column: amount
    severity: error

  - name: currency_is_iso_3
    type: regex
    column: currency
    params:
      pattern: "^[A-Z]{3}$"
    severity: error

  - name: transaction_type_known
    type: in_set
    column: transaction_type
    params:
      values: ["INVOICE", "PAYMENT", "REFUND", "ADJUSTMENT"]
    severity: warning
```

Ejecución

```python
df = pl.DataFrame(records)
rules = load_rules_yaml("src/data_quality/rules/transactions.yaml")
results = run_rules(df, rules, suite="commercial_ingestion")
assert_no_errors(results)   # raises si hay severity=error con passed=false
```

Cada ejecución emite `ldp_dq_checks_total{suite, result=passed|failed}` a Prometheus.

Line 3 — Great Expectations

GE aporta lo que el rule engine casero no cubre bien:

- **Expectations estadísticas**: `expect_column_mean_to_be_between`, `expect_column_kl_divergence_to_be_less_than`.
- **Data Docs**: HTML estático auto-generado con el estado de cada suite — útil para compartir con stakeholders no-técnicos.
- **Profiling**: `UserConfigurableProfiler` arma una suite inicial inspeccionando los datos.
- **Checkpoints**: pipelines GE con actions (Slack notification, S3 upload de docs).

Integración

src/data_quality/ge_integration.py provee helpers para:

- `build_expectation_suite( ... )` desde un dict programático.
- `validate_dataframe(df, suite)` ejecuta la suite en un DataFrame.

Las suites de producción viven en un directorio `gx/` que GE gestiona, separado del rule engine.

Quarantine

Registros que fallan validación no se tiran a la basura; se escriben a S3 quarantine con:

- El registro original.
- El nombre del schema que falló.
- La lista de errores Pydantic (campo, tipo, mensaje).
- Timestamp de rechazo.

```
{
  "raw": { "doc_id": "", "date": "not-a-date", ... },
  "schema": "LegalDocumentSchema",
  "errors": [
    { "loc": ["document_date"], "msg": "Input should be a valid date", ... }
  ],
  "rejected_at": "2026-04-24T03:15:00Z"
}
```

Un proceso de triaje semanal revisa los rechazos y retroalimenta a los productores de datos.

Métricas y alertas

Métrica	Alerta
`ldp_records_quarantined_total` rate	>5% del total ingerido en 10min → page
`ldp_dq_checks_total{result="failed"}` rate	>1/min por 5min → warning
`ldp_pipeline_errors_total`	cualquier valor → warning
Deriva detectada por GE	notification Slack

Data contracts (roadmap)

Fase siguiente: **data contracts** negociados entre productor y consumidor.

- Esquema JSON Schema / Protobuf en schema registry.
- Breaking changes requieren version bump y aprobación del consumidor.
- CI del productor corre tests contra consumer-defined expectations antes de publish.

Librerías candidatas: Buf (Protobuf), Confluent Schema Registry, `dbt data contracts`.